

Perturb-seq simulation pipeline

End-to-end walkthrough of how the simulator turns a GRN and a set of observed control cells into a simulated perturb-seq dataset. Two layers:

- **Layer 1 — Biological.** Compute a per-gene log fold change for each KO from the GRN cascade.
- **Layer 2 — Data-generating.** Turn that log fold change into per-cell counts.

All code lives in `SRC/PerturbSeqSimulator.py` (and `SRC/GenerateGRN.py` for the GRN construction).

Layer 1 — Per-gene log_fc for each KO

Ingredients

- **W matrix**, shape `(n_genes, n_programs)` — how much each gene’s expression responds to each program’s activity. Sampled from per-program Gaussians (~70% positive entries by default — a gene typically activates a program). Non-zero only for program-member genes.
- **V matrix**, shape `(n_programs, n_genes)` — how much each program’s activity changes when each gene is knocked out. Non-zero where **W** is; sign inverted from **W** per pair — a gene that activates a program has negative KO effect on that program (KO silences the program).
- $|V| < |W|$ is enforced for every connected gene-program pair.

Primary effect of KO of gene *g*

The KO removes gene *g*’s contribution from every program it’s in. Column *g* of **V** directly encodes it:

```
primary_delta_h = V[:, g] * perturbation_strength      # (n_programs,)
```

`delta_h[p]` is the change in program *p*’s activity. Non-zero only for programs containing gene *g*.

Secondary cascade

With `n_propagation_steps = 2` (default), one extra round runs. The primary shift in program activity perturbs other genes through **W**; those gene changes feed back to programs through **V**, attenuated by `damping_factor = 0.5`:

```
delta_g          = W @ delta_h          # programs -> genes
delta_g[g]       = 0                   # KO gene is silenced
delta_h_next     = V @ delta_g * damping_factor  # genes -> programs
total_delta_h += delta_h_next
```

Conceptually: “KO perturbs program *A* → affects *A*’s genes → some of those are also in program *B* → feeds back into *B*.” Damping < 1 keeps this from exploding.

Per-gene log fold change

Once the total program shift is known, map back to genes one more time:

```
log_fc = W @ total_delta_h              # (n_genes,), natural log
log_fc = clip(log_fc, ±max_log2_fc * ln(2))  # cap runaway values, default ±5 log2 = ±32x
```

Stored as `self.effects[gene_name]['log_fold_change']`. The ground-truth matrix `get_ground_truth_log2fc()` just divides this by $\ln(2)$ for every KO, sets the KO gene’s own entry to `-inf` (complete silencing), and returns an `(n_KO, n_genes)` DataFrame.

Layer 2 — From log_fc to counts

For each KO:

Step 1 — Sample baseline control cells

With replacement, `n_cells` rows from the observed control count matrix:

```
idx      = rng.choice(n_control, size=n_cells, replace=True)
baseline = control_counts[idx, :]                # (n_cells, n_genes), integer
```

Step 2 — Log1p-shift with biological noise

In log1p space, shift every gene's count by its `log_fc` and add zero-mean Gaussian noise for cell-to-cell biological variability:

```
y_pert = log1p(baseline) + log_fc[None, :]
y_pert += Normal(0, noise_std)                # default noise_std = 0.5
y_pert[:, ko_gene] = 0                       # KO gene silenced
```

The key identity

$$\log(X_{\text{pert}} + 1) = \log(X_{\text{ctrl}} + 1) + \log_{\text{fc}} + \text{biological_noise}$$

So $E[\log(X_{\text{pert}} + 1) - \log(X_{\text{ctrl}} + 1)] = \log_{\text{fc}}$. That's why $\text{mean}(\log1p(X_{\text{pert}})) - \text{mean}(\log1p(X_{\text{ctrl}}))$ is an unbiased estimator of `log_fc` — the GT is recoverable from the data by construction.

Step 3 — Invert back to count space

```
counts = expm1(y_pert)                        # float, range (-1, inf)
```

Floats rather than integers, because rounding or clamping would break the identity for cells where `X_ctrl` = 0 and `log_fc` < 0 (sub-2× downregulation collapses to 0 under rounding).

Step 4 — NB sampling for technical noise (optional)

If `nb_sampling` = `True`, draw integer counts from NB around the float mean, using the per-gene dispersion estimated from controls (floored at `min_dispersion` = 5 to avoid runaway overdispersion):

```
r      = dispersion[None, :]                  # per-gene shape param
mu     = max(counts, 0)
p      = r / (r + mu)
counts = NB(r, p)                             # integer, mean=mu, var=mu + mu^2/r
counts[:, ko_gene] = 0
```

This gives real-looking integer counts with the characteristic count-data mean-variance curve — Poisson-like at low `mu`, overdispersed at high `mu` — at the cost of a small Jensen-inequality bias in the log1p-mean-diff estimator (~0.05–0.15 log2 under the current dispersion floor).

Turn it off (`nb_sampling` = `False`) for float output and a perfect GT-empirical diagonal.

Assembly (`simulate_all_perturbations`)

For each requested KO gene, call the above to get an (`n_cells_per_pert`, `n_genes`) block; vstack all perturbed blocks with the original control cells; wrap in an `AnnData` with:

- `obs['perturbation']` — KO gene name or 'control'
- `obs['is_control']` — boolean

- `uns['ground_truth_log2fc']` — GT matrix (via `get_ground_truth_log2fc`)
- `varm['W'], uns['V']` — GRN matrices for reference

Noise layers and their roles

Layer	Source of variance	Controlled by	What it simulates
Gaussian in log1p	<code>noise_std</code>	<code>__init__(noise_std=...)</code>	cell-to-cell biological heterogeneity
NB sampling	per-gene dispersion	<code>__init__(nb_sampling=..., min_dispersion=...)</code>	technical / capture noise of droplet scRNA-seq

Useful presets

Preset	<code>noise_std</code>	<code>nb_sampling</code>	Characteristics
Deterministic	0.0	False	Pure log1p-shift. GT recoverable exactly. Volcano p-values saturate trivially.
Gaussian-only (default, matches GT)	0.5	False	Float counts, realistic p-value gradient, GT-empirical diagonal near-perfect both signs.
Two-stage realistic	0.5	True	Integer counts for DESeq2/glmGamPoi. Jensen bias + zero-clamp asymmetry attenuate the diagonal.

Key parameters cheat sheet

Parameter	Default	Effect
<code>perturbation_strength</code>	1.0	Scales the primary KO effect. 1.0 = full knockout.
<code>damping_factor</code>	0.5	Attenuation at each secondary propagation step.
<code>n_propagation_steps</code>	2	Rounds of V→W propagation. 1 = primary only, 2+ = includes secondary cascade.
<code>max_log2_fc</code>	5.0	Symmetric clip on per-gene log2 fold change. Default ±32x, physiologically generous.
<code>noise_std</code>	0.5	Stddev of Gaussian noise added in log1p space (biological variability).
<code>nb_sampling</code>	False	Whether to layer NB sampling on top of the deterministic/Gaussian output (integer counts).
<code>min_dispersion</code>	5.0	Floor on per-gene NB dispersion τ . Higher means less overdispersion, cleaner signal.